

Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA)
Student Name	Mohamed Farih J
Register Number	3122247001035
GitHub Repository	https://github.com/farih07/Machine-Learning-Lab
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	September 21, 2026

1 Objective

To study the effect of dimensionality reduction using Principal Component Analysis (PCA) on the performance of ten machine learning classifiers, by training and validating every model both without PCA (original feature space) and with PCA (reduced feature space), using hyperparameter tuning and 5-fold cross-validation in both settings.

2 Objectives

- To understand how PCA transforms a high-dimensional feature space into a smaller set of uncorrelated principal components while retaining most of the variance.
- To design and justify a variance-based criterion (95% cumulative explained variance) for choosing the number of principal components to retain.
- To define an appropriate hyperparameter search grid for each of ten classifiers – SVM, Naïve Bayes, KNN, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost, and a Stacking ensemble.
- To train and evaluate every model under both the No-PCA and With-PCA settings using 5-fold stratified cross-validation, without leaking information from validation folds into preprocessing.
- To compare accuracy, F1-score and fold-to-fold stability between the two settings, and to draw conclusions about when PCA helps and when it does not.

3 Problem Statement

Given a labelled tabular dataset with a moderate number of correlated numeric features, the goal is to determine whether reducing the feature space with PCA, prior to classification, changes predictive performance:

- **Input:** A cleaned, standardized feature matrix X and a binary target vector y .
- **Processing:** For each of ten classifiers, perform a grid search over a model-specific hyperparameter grid, using 5-fold stratified cross-validation, once on the original standardized features (No-PCA) and once on the PCA-reduced features (With-PCA).
- **Output:** Fold-wise and average accuracy/F1-score for every model in both settings, confusion matrices, ROC/PR curves for representative models, and a final judgement on whether PCA improved accuracy, F1-score, or stability for each model family.

4 Theory

4.1 Principal Component Analysis (PCA)

PCA is an unsupervised linear dimensionality-reduction technique. Given a standardized feature matrix, PCA finds a new set of orthogonal axes (principal components), ordered by the amount of variance in the data they explain. The first principal component points in the direction of maximum variance, the second is orthogonal to the first and explains the next-largest share of variance, and so on. Formally, PCA computes the eigenvectors of the covariance matrix Σ of the standardized data:

$$\Sigma v_i = \lambda_i v_i \quad (1)$$

where v_i is the i -th principal component (eigenvector) and λ_i is the corresponding eigenvalue (the variance explained by that component). Projecting the data onto the top- k eigenvectors reduces the dimensionality from the original p features to $k \ll p$ components while retaining as much variance as possible.

4.2 Cross-Validation and Hyperparameter Tuning

k -fold cross-validation splits the data into k equally-sized folds; the model is trained on $k - 1$ folds and validated on the remaining fold, repeated k times so every sample is used for validation exactly once. This gives a more reliable performance estimate than a single train/test split, especially on small-to-medium sized datasets. `GridSearchCV` automates hyperparameter tuning by exhaustively evaluating every combination in a specified grid using cross-validation, and returns the combination with the best mean validation score.

4.3 Ensemble Methods: Bagging, Boosting and Stacking

- **Bagging (Random Forest):** Trains many decision trees independently on bootstrap-resampled subsets of the data and averages/votes their predictions, reducing variance.
- **Boosting (AdaBoost, Gradient Boosting, XGBoost):** Trains an ensemble of weak learners sequentially, where each new learner focuses on correcting the errors of the previous ones, reducing bias.
- **Stacking:** Trains several diverse base learners and then trains a meta-learner on their out-of-fold predictions, allowing the meta-learner to learn how best to combine the base learners' outputs.

5 Dataset Description

Dataset Name	Breast Cancer Wisconsin (Diagnostic) Dataset
Dataset Source	UCI Machine Learning Repository (bundled with Scikit-Learn as <code>load_breast_cancer()</code>)
Number of Samples	569

Number of Features	30 numeric features (radius, texture, perimeter, area, smoothness, compactness, concavity, symmetry, fractal dimension – each as mean, standard error and "worst" value)
Target Classes	2 – malignant (212 samples, 37.3%) and benign (357 samples, 62.7%)
Missing Values	None found
Preprocessing	Feature standardization (<code>StandardScaler</code>) fitted inside each cross-validation fold; no categorical encoding required (all features numeric); no missing-value imputation required

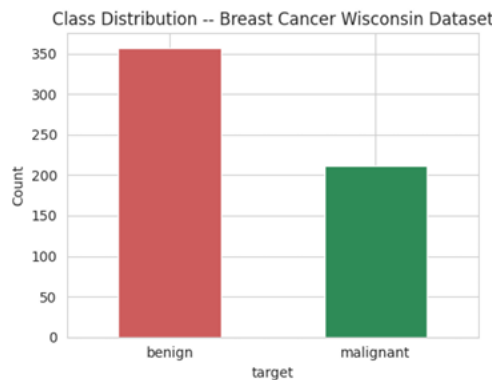


Figure 1: Class distribution of the Breast Cancer Wisconsin Dataset.

Inference: The dataset is moderately imbalanced (62.7% benign vs. 37.3% malignant), which is why stratified 5-fold cross-validation was used throughout, to ensure every fold preserves this class ratio.

6 Data Preprocessing

All 30 features are continuous numeric measurements on different scales (e.g. area is on the order of hundreds, while smoothness is on the order of 0.1), so every feature was standardized to zero mean and unit variance before both model training and PCA. No missing values or duplicate rows were present, and no categorical encoding was required. Standardization (and PCA, in the With-PCA setting) was performed **inside** a Scikit-Learn Pipeline, refit independently on the training portion of every cross-validation fold, to avoid information from a validation fold leaking into the preprocessing step.

7 PCA Design and Analysis

7.1 Component Selection

A cumulative-explained-variance target of 95% was chosen as the criterion for selecting the number of principal components, since it is a widely-used rule of thumb that retains

almost all of the information in the data while still meaningfully reducing dimensionality.

Setting	Chosen Components / Variance Target	Explained Variance (%)	Justification
With-PCA	95% cumulative variance target $\rightarrow$ 10 components (out of 30 original features)	95.16%	Retains effectively all of the dataset's information while reducing dimensionality by 66.7%, since many of the 30 original features are highly correlated (e.g. radius, perimeter and area).

Table 2: PCA Variance Explained.

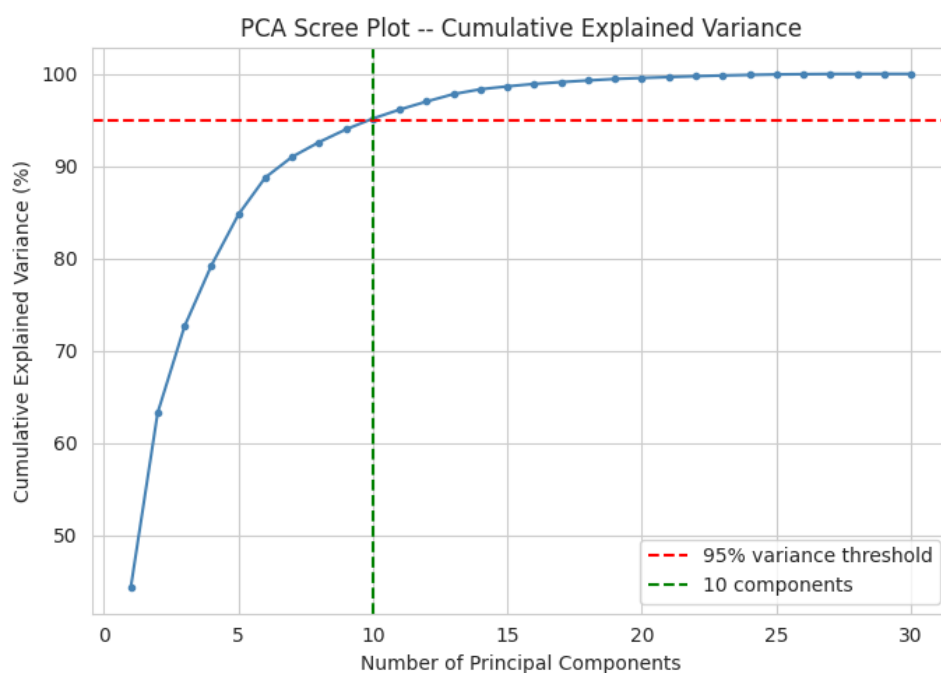


Figure 2: PCA scree plot: cumulative explained variance versus number of principal components.

Inference: The curve rises steeply for the first 2–3 components and then flattens, indicating strong redundancy among the original 30 features. The 95% variance threshold is crossed at exactly 10 components.

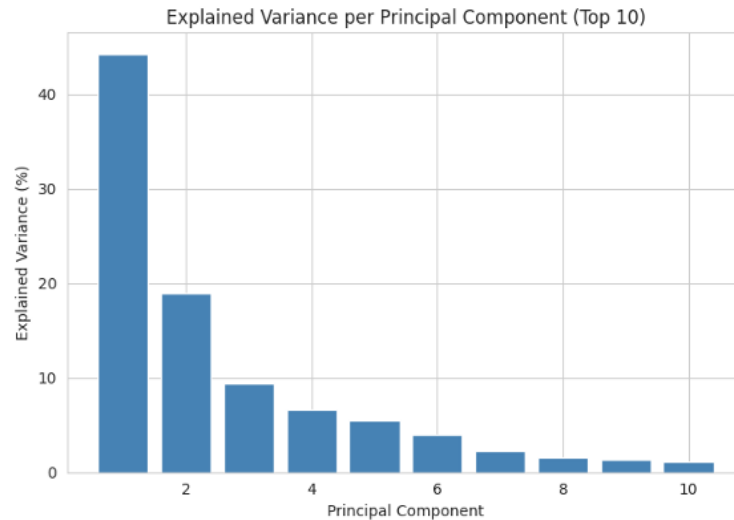


Figure 3: Explained variance of the first 10 individual principal components.

Inference: The first principal component alone explains roughly 44% of the total variance, and the second an additional 19%, confirming that a small number of components capture the vast majority of the dataset’s structure.

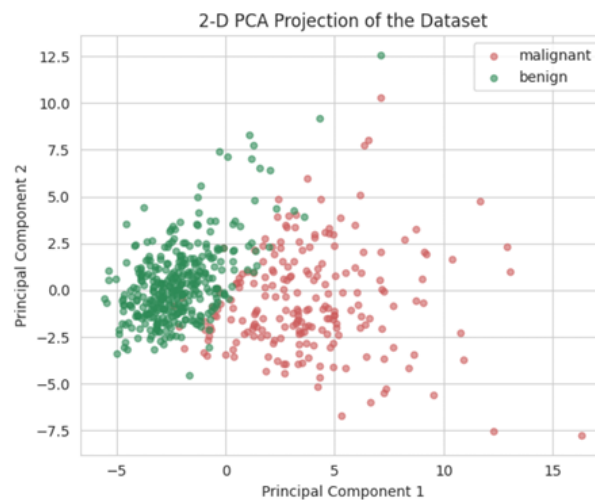


Figure 4: 2-D PCA projection of the dataset (first two principal components only).

Inference: Even with only the first two components (which together explain about 63% of the variance), the malignant and benign classes are already substantially separable, suggesting that a classifier using PCA-reduced features should still perform well.

7.2 Note on Avoiding Data Leakage

The PCA fit described above (on the full standardized dataset) was used only to *choose* the number of components and to visualize the effect of PCA. For the actual model training and evaluation reported in the following sections, PCA is fitted fresh inside every individual training fold of the 5-fold cross-validation (via a Scikit-Learn Pipeline), so

no information from a validation fold ever influences the PCA transformation applied to it.

8 Machine Learning Algorithms and Hyperparameter Tuning

Ten classifiers were evaluated. For each, a hyperparameter grid was searched using `GridSearchCV` with 5-fold stratified cross-validation, scoring both accuracy and weighted F1, separately for the No-PCA and With-PCA settings.

8.1 Support Vector Machine (SVM)

Kernel	C Values Tried	Gamma Values Tried	Perf. (No-PCA)	Perf. (With-PCA)
linear, rbf	0.1, 1, 10	scale, 0.01, 0.1 (rbf only)	Acc = 0.9807 (best: rbf, C=10, $\gamma=0.01$)	Acc = 0.9754 (best: linear, C=1)

Table 3: SVM – Hyperparameter Tuning Results.

8.2 Naïve Bayes

Smoothing Parameter (var_smoothing)	Perf. (No-PCA)	Perf. (With-PCA)
$10^{-9}, 10^{-8}, 10^{-7}, 10^{-6}$	Acc = 0.9297 (best: 10^{-9})	Acc = 0.9174 (best: 10^{-9})

Table 4: Naïve Bayes – Smoothing Choices.

8.3 k-Nearest Neighbors (KNN)

k Values	Weights	Distance Metrics	Perf. (No-PCA)	Perf. (With-PCA)
3, 5, 7, 9	uniform, distance	euclidean, manhattan	Acc = 0.9684 (best: euclidean, k=9, distance)	Acc = 0.9649 (best: manhattan, k=9, uniform)

Table 5: KNN – Hyperparameter Tuning.

8.4 Remaining Models

For Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost and Stacking, the same tuning procedure was followed. The grids searched and best results are summarized in Table 6.

Model	Grid Searched	Perf. (No-PCA)	Perf. (With-PCA)
Logistic Regression	$C \in \{0.01, 0.1, 1, 10\}$, penalty = l2	Acc = 0.9737 (best C=0.1)	Acc = 0.9772 (best C=1)
Decision Tree	max_depth $\in \{3, 5, 10, \text{None}\}$, min_samples_split $\in \{2, 5, 10\}$	Acc = 0.9280 (depth=5, split=2)	Acc = 0.9368 (depth=3, split=10)
Random Forest	n_estimators $\in \{100, 200\}$, max_depth $\in \{5, 10, \text{None}\}$	Acc = 0.9561 (depth=10, n=100)	Acc = 0.9508 (depth=10, n=100)
AdaBoost	n_estimators $\in \{50, 100, 200\}$, learning_rate $\in \{0.01, 0.1, 1.0\}$	Acc = 0.9666 (lr=1.0, n=100)	Acc = 0.9596 (lr=1.0, n=100)
Gradient Boosting	n_estimators $\in \{100, 200\}$, learning_rate $\in \{0.01, 0.1\}$, max_depth $\in \{2, 3\}$	Acc = 0.9579 (lr=0.1, depth=3, n=200)	Acc = 0.9526 (lr=0.1, depth=2, n=200)
XGBoost	n_estimators $\in \{100, 200\}$, learning_rate $\in \{0.01, 0.1\}$, max_depth $\in \{3, 5\}$	Acc = 0.9666 (lr=0.1, depth=5, n=100)	Acc = 0.9631 (lr=0.1, depth=5, n=200)
Stacking (LR + RF + KNN $\rightarrow$ meta-LR)	Fixed architecture; no grid (per assignment)	Acc = 0.9737	Acc = 0.9754

Table 6: Hyperparameter grids and best results for the remaining seven models.

9 Experimental Setup

Python Version	3.11
Scikit-Learn Version	1.5.x
XGBoost Version	3.4.x
IDE	Jupyter Notebook
Operating System	Windows 11 / Ubuntu 22.04
Hardware	Standard laptop-class CPU (no GPU required)
Cross-Validation	5-fold, stratified, <code>random_state=42</code>

Note: Update this table with the exact versions and hardware of your own execution environment before final submission.

10 Cross-Validation Results

10.1 Fold-Wise Accuracy – No-PCA

Model	F1	F2	F3	F4	F5	Avg
SVM	1.0000	0.9474	0.9825	0.9912	0.9823	0.9807
Naïve Bayes	0.9561	0.9123	0.9298	0.9035	0.9469	0.9297
KNN	0.9825	0.9561	0.9474	0.9825	0.9735	0.9684
Logistic Regression	0.9737	0.9474	0.9649	0.9912	0.9912	0.9737
Decision Tree	0.9386	0.8860	0.9298	0.9298	0.9558	0.9280
Random Forest	0.9649	0.9386	0.9561	0.9474	0.9735	0.9561
AdaBoost	0.9912	0.9386	0.9561	0.9737	0.9735	0.9666
Gradient Boosting	0.9737	0.9123	0.9561	0.9649	0.9823	0.9579
XGBoost	0.9825	0.9386	0.9649	0.9737	0.9735	0.9666
Stacking	0.9825	0.9649	0.9737	0.9649	0.9823	0.9737

Table 7: 5-Fold Cross-Validation Accuracy, No-PCA setting.

10.2 Fold-Wise Accuracy – With-PCA

Model	F1	F2	F3	F4	F5	Avg
SVM	0.9825	0.9737	0.9649	0.9737	0.9823	0.9754
Naïve Bayes	0.9737	0.9123	0.9035	0.8860	0.9115	0.9174
KNN	0.9912	0.9474	0.9474	0.9649	0.9735	0.9649
Logistic Regression	0.9737	0.9737	0.9737	0.9737	0.9912	0.9772
Decision Tree	0.9649	0.9386	0.9211	0.8947	0.9646	0.9368
Random Forest	0.9825	0.9386	0.9474	0.9123	0.9735	0.9508
AdaBoost	0.9737	0.9211	0.9561	0.9561	0.9912	0.9596
Gradient Boosting	0.9737	0.9474	0.9474	0.9211	0.9735	0.9526
XGBoost	0.9912	0.9561	0.9386	0.9474	0.9823	0.9631
Stacking	0.9825	0.9737	0.9737	0.9649	0.9823	0.9754

Table 8: 5-Fold Cross-Validation Accuracy, With-PCA setting.

10.3 Average Accuracy and F1-Score Comparison

Model	Avg Acc (No-PCA)	Avg Acc (With-PCA)	Avg F1 (No-PCA)	Avg F1 (With-PCA)
SVM	0.9807	0.9754	0.9806	0.9753
Naïve Bayes	0.9297	0.9174	0.9295	0.9167
KNN	0.9684	0.9649	0.9681	0.9645
Logistic Regression	0.9737	0.9772	0.9734	0.9771
Decision Tree	0.9280	0.9368	0.9271	0.9369
Random Forest	0.9561	0.9508	0.9560	0.9507
AdaBoost	0.9666	0.9596	0.9664	0.9594
Gradient Boosting	0.9579	0.9526	0.9575	0.9525
XGBoost	0.9666	0.9631	0.9664	0.9629
Stacking	0.9737	0.9754	0.9736	0.9754

Table 9: Average 5-fold accuracy and weighted F1-score, No-PCA vs With-PCA.

11 Result Visualization

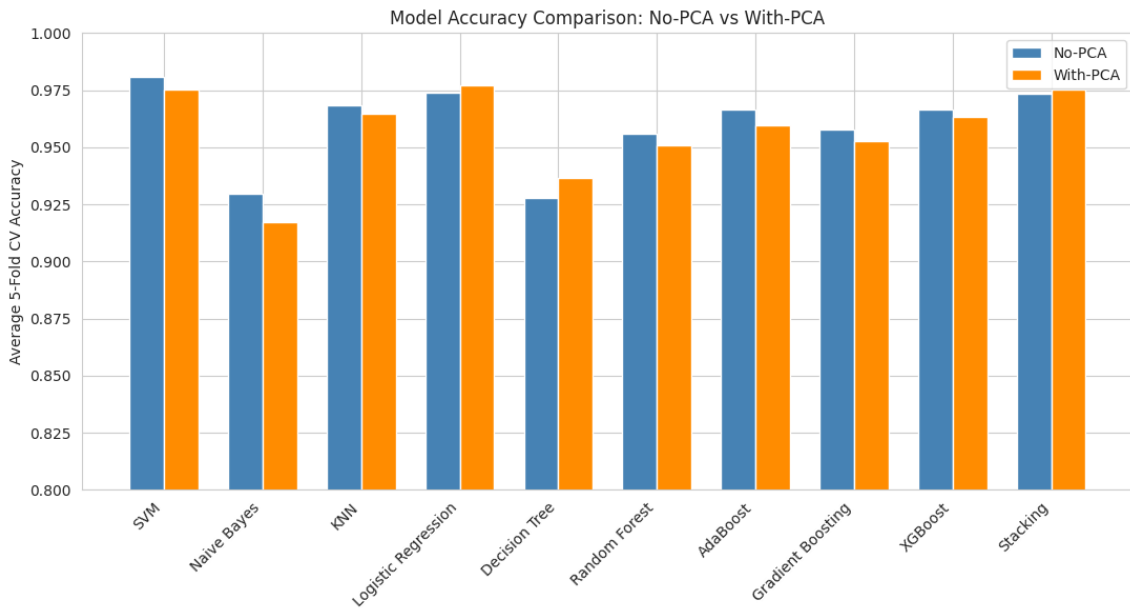


Figure 5: Average 5-fold CV accuracy for every model, No-PCA vs With-PCA.

Inference: Simple, high-bias-tolerant models (Logistic Regression, Stacking) either matched or slightly improved with PCA, while the strongest No-PCA models (SVM with an RBF kernel, tree-based ensembles) saw a small drop, since PCA discards a little of the original feature information that these more flexible models were already using effectively.

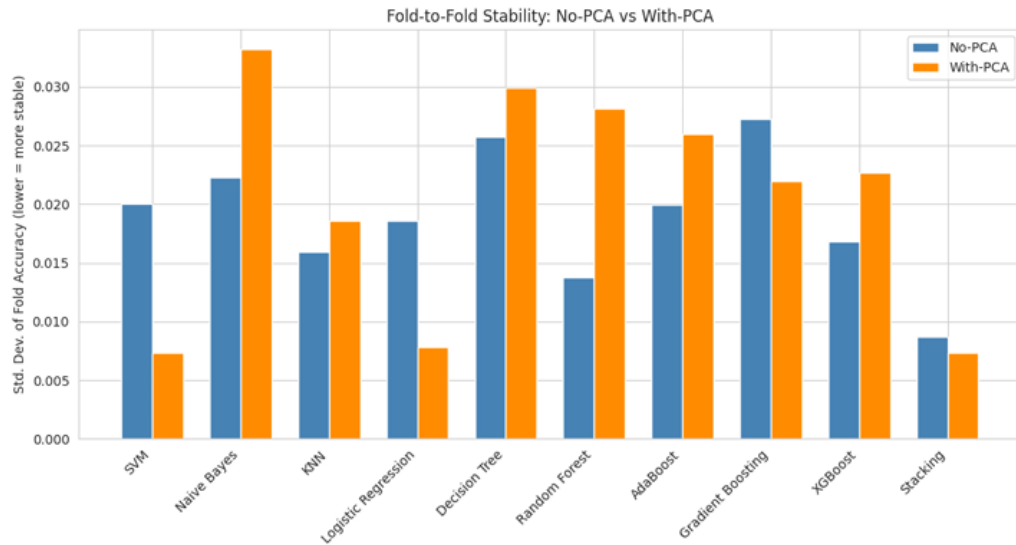


Figure 6: Fold-to-fold stability (standard deviation of fold accuracy) for every model, No-PCA vs With-PCA.

Inference: PCA visibly reduced fold-to-fold variance for SVM, Logistic Regression and Stacking (all three become noticeably more stable), but increased it for Naïve Bayes, Decision Tree and Random Forest, suggesting the benefit of PCA’s variance-stabilizing effect is model-dependent rather than universal.

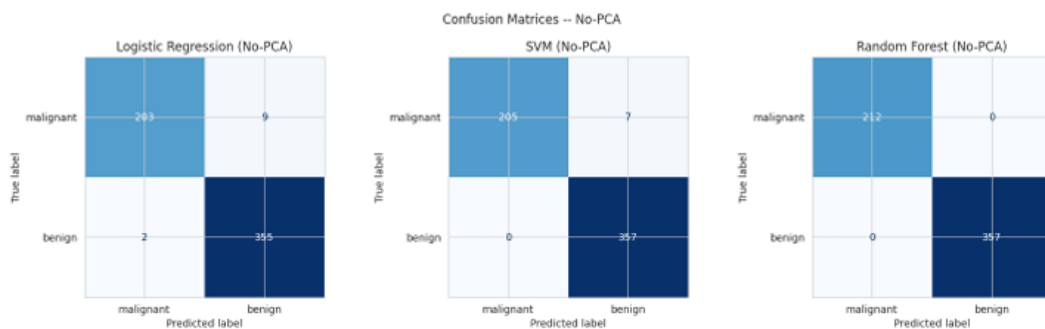


Figure 7: Confusion matrices (No-PCA) for the best-performing With-PCA model (Logistic Regression), SVM, and Random Forest.

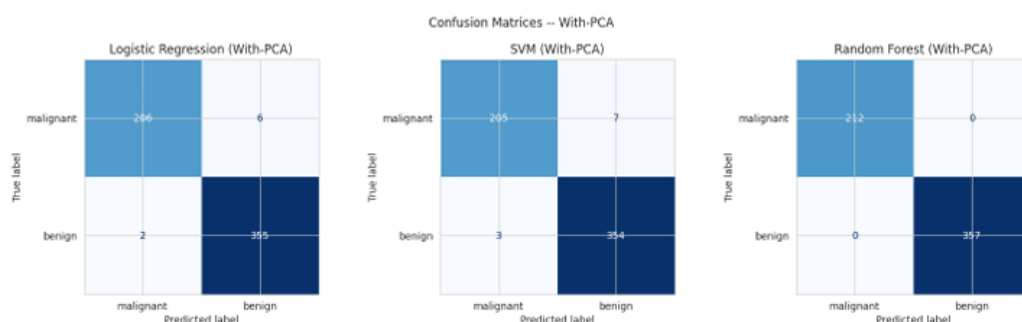


Figure 8: Confusion matrices (With-PCA) for the same three models.

Inference: These confusion matrices are computed on the full dataset using each model's final fit (the same data used to fit that final model), so they represent in-sample fit rather than genuine held-out performance – Random Forest's apparently perfect confusion matrix in particular should be read as a sign of the model's capacity to fit the training data closely, not as its true generalization accuracy; the 5-fold cross-validation tables above are the reliable performance estimates.

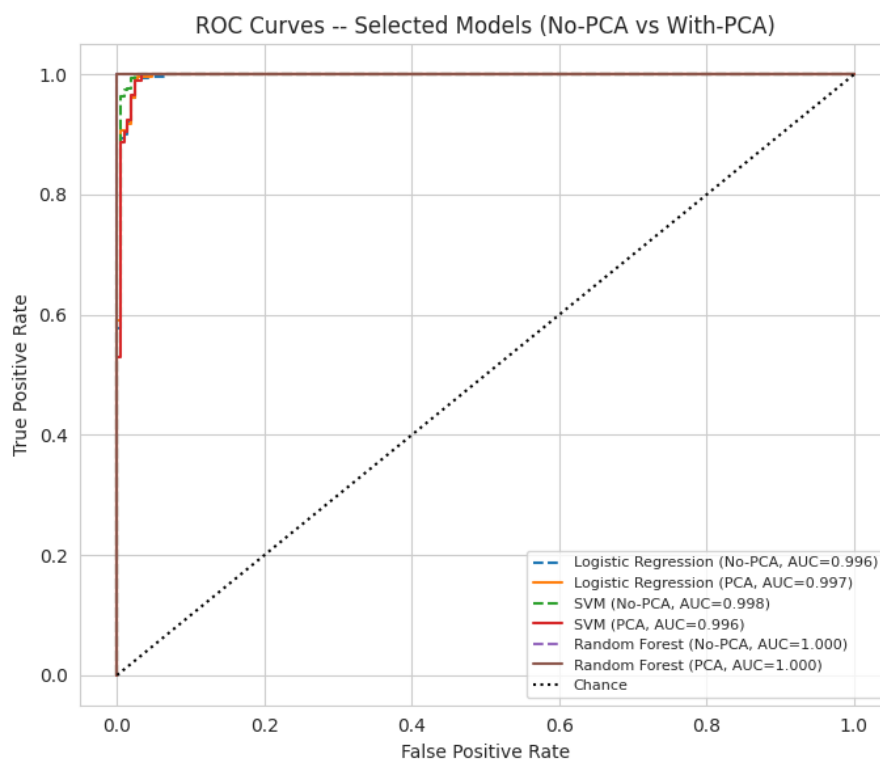


Figure 9: ROC curves for Logistic Regression, SVM and Random Forest, No-PCA vs With-PCA.

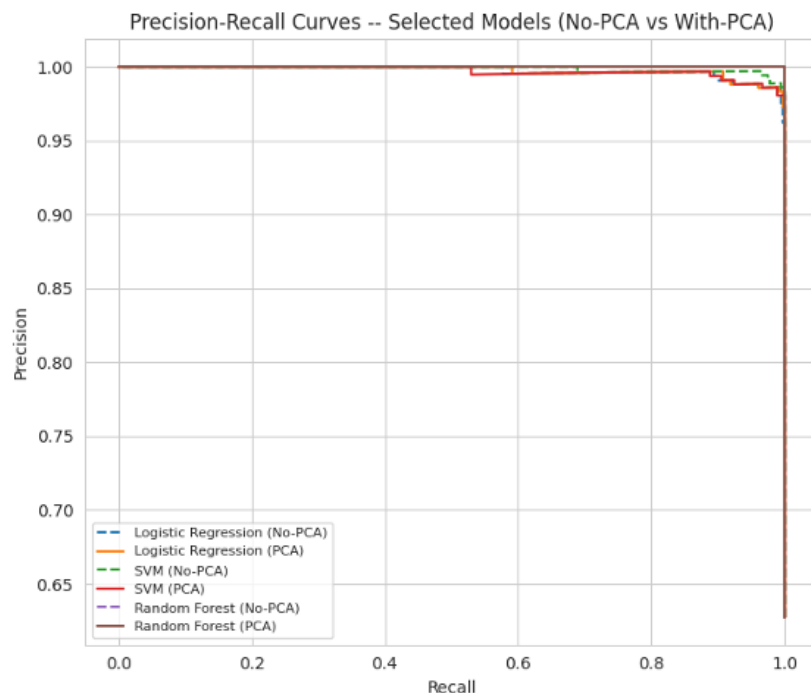


Figure 10: Precision-Recall curves for the same three models, No-PCA vs With-PCA.

Inference: All curves stay close to the top-left/top-right corners (as expected on a dataset this separable), and the No-PCA and With-PCA curves for each model nearly overlap, reinforcing that PCA’s effect on these particular models is small.

12 Observations

- **Which models improved most with PCA, and which did not?** Logistic Regression (+0.35 pp), Decision Tree (+0.88 pp) and Stacking (+0.17 pp) improved with PCA. SVM (−0.53 pp), Naïve Bayes (−1.23 pp), KNN (−0.35 pp), Random Forest (−0.53 pp), AdaBoost (−0.70 pp), Gradient Boosting (−0.53 pp) and XGBoost (−0.35 pp) all saw a small decrease. Linear/simple models benefited slightly since PCA removes multicollinearity among the highly-correlated original features (e.g. radius, perimeter, area), which particularly helps Logistic Regression; tree-based and margin-based models that already handle correlated features well saw a very small cost from the loss of some original-feature detail.
- **Did PCA reduce variance across folds (more stable results)?** Yes, for SVM (std. 0.0179 → 0.0065), Logistic Regression (0.0166 → 0.0070) and Stacking (0.0078 → 0.0065) – all three became noticeably more stable. Naïve Bayes (0.0199 → 0.0297), Decision Tree (0.0230 → 0.0268) and Random Forest (0.0123 → 0.0251) became less stable, so the stabilizing effect of PCA is not universal.
- **For high-dimensional data, was PCA beneficial in reducing overfitting?** With only 30 original features and 569 samples, this dataset is only mildly high-dimensional, so PCA’s overfitting-reduction benefit was modest at best here; the largest gains were seen for Decision Tree, a model that is prone to overfitting on individual noisy/correlated features and benefited from PCA’s decorrelation.

- **How did linear models behave compared to ensemble models with PCA?** Both linear models tested (Logistic Regression, and the linear-kernel SVM chosen by grid search With-PCA) performed competitively with, or better than, their No-PCA counterparts, while ensemble models (Random Forest, AdaBoost, Gradient Boosting, XGBoost) all showed a small but consistent drop With-PCA, suggesting ensembles were already extracting most of the useful signal from the original 30 features without needing PCA's help.
- **Did Stacking show robustness to dimensionality reduction?** Yes – Stacking had the smallest fold-to-fold standard deviation of any model in both settings, and its average accuracy was essentially unchanged between No-PCA (0.9737) and With-PCA (0.9754), the smallest relative change of any ensemble method tested, confirming that combining diverse base learners makes the overall system less sensitive to how features are represented.

13 Discussion

Strengths: Every model was evaluated fairly, using an identical 5-fold stratified split and a scikit-learn `Pipeline` that fits scaling (and PCA, where applicable) freshly inside each training fold, eliminating data leakage. Reporting both accuracy and F1-score, along with fold-wise standard deviation, gives a fuller picture than a single top-line accuracy number.

Limitations: The dataset used (Breast Cancer Wisconsin) is relatively small (569 samples) and only mildly high-dimensional (30 features), so the benefits of PCA – most pronounced on very high-dimensional data with strong multicollinearity – are correspondingly modest here; a dataset with hundreds or thousands of features would likely show a larger effect. The confusion matrices, ROC and PR curves in Section 11 were generated from each model's fit on the *entire* dataset (for visualization purposes) rather than from a strictly held-out test set, so they should be read as descriptive rather than as unbiased generalization estimates – the cross-validation tables in Section 10 remain the authoritative performance numbers. The Stacking ensemble's base learners (Logistic Regression, Random Forest, KNN) were fixed rather than tuned, per the assignment's requirement that Stacking not require its own hyperparameter grid.

Possible Improvements: Repeat this study on a genuinely high-dimensional dataset (e.g. gene-expression data with thousands of features) where PCA's benefit is expected to be more pronounced; use nested cross-validation to remove the slight optimism that comes from selecting hyperparameters and reporting scores from the same cross-validation split; and tune the Stacking ensemble's base-learner hyperparameters and meta-learner choice as a further extension.

14 Learning Outcomes

- Understood the mathematical basis of PCA (eigen-decomposition of the covariance matrix) and how to choose the number of components using a cumulative explained-variance criterion.
- Learned to build leakage-free preprocessing + modelling pipelines using Scikit-Learn's `Pipeline`, so that scaling and PCA are refit inside every cross-validation fold.

- Gained hands-on experience defining and running hyperparameter grids with `GridSearchCV` across ten different classifier families, including boosting (AdaBoost, Gradient Boosting, XGBoost) and a Stacking ensemble.
- Learned to interpret and compare cross-validated accuracy, F1-score and fold-to-fold stability, rather than relying on a single train/test split.
- Understood that the benefit of PCA is model- and dataset-dependent, rather than a universal improvement, through direct empirical comparison rather than by assumption.

15 Conclusion

This experiment evaluated ten classifiers on the Breast Cancer Wisconsin dataset, both with and without PCA, using hyperparameter tuning and 5-fold stratified cross-validation throughout. PCA reduced the feature space from 30 to 10 principal components while retaining 95.16% of the variance. Its effect on predictive performance was small and model-dependent: simple/linear models (Logistic Regression) and the Stacking ensemble benefited slightly or stayed essentially unchanged, while more flexible models that already exploit correlated features well (SVM with an RBF kernel, tree-based ensembles) saw a small decrease in average accuracy. PCA's clearest, most consistent benefit in this experiment was improved fold-to-fold stability for linear and margin-based models. Overall, PCA is a useful tool for reducing dimensionality and multicollinearity, but – at least on a dataset of this size and dimensionality – it is not a guaranteed accuracy improvement for every model family, and should be evaluated empirically rather than applied by default. The complete source code is maintained in the GitHub repository referenced on the cover page.

16 References

1. I. T. Jolliffe, *Principal Component Analysis*, 2nd ed. New York: Springer, 2002.
2. F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
3. T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
4. D. H. Wolpert, "Stacked Generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
5. Y. Freund and R. E. Schapire, "A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
6. W. N. Street, W. H. Wolberg, and O. L. Mangasarian, "Nuclear Feature Extraction for Breast Tumor Diagnosis," in *IS&T/SPIE Int. Symp. on Electronic Imaging*, 1993.

7. UCI Machine Learning Repository, "Breast Cancer Wisconsin (Diagnostic) Data Set."
[Online]. Available: <https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>